

山东大学 网络空间安全学院

创新创业实践课 同态加密实验作业

组员

李瑞佳

岳智宇

薛俊东

张金浩

学

院

网络空间安全学院

完成时间

2026 年 7 月 25 日

目录

一 引言	1
二 BFV 全同态加密方案原理	1
2.1 多项式环	1
2.2 密钥体系	1
2.3 加解密	2
2.4 同态运算	2
三 打包-旋转-累加策略详解	2
3.1 策略动机	2
3.2 数据布局与展开	3
3.3 旋转对齐的数学原理	3
四 代码逐行详解	4
4.1 辅助函数 print_matrix	4
4.2 plaintext_convolution: 明文参考实现	4
4.3 main 函数: 明文数据定义	5
4.4 main 函数: BFV 参数配置	6
4.5 main 函数: 密钥生成	7
4.6 main 函数: 数据打包与加密	8
4.7 main 函数: 密文卷积核心循环	8
4.8 main 函数: 解密与结果提取	10
4.9 main 函数: 步骤 7——正确性验证	11
五 实验结果与分析	11
5.1 手动验算	11
5.2 密文验证结果	12

六 旋转次数优化：是否达到理论最小值？	13
6.1 当前实现的旋转次数	13
6.2 理论下界分析	13
6.2.1 卷积核的可分离性	13
6.2.2 水平通道：1 次旋转	14
6.2.3 垂直通道：2 次旋转	14
6.2.4 总计与分析	15
6.3 3 是否为绝对下界？	15
6.4 可分离核法的同态运算总量	15
6.5 可分离核法的伪代码	16
6.6 小结	16
七 总结	17

一 引言

全同态加密允许在加密数据上直接进行计算而无需解密。在隐私保护机器学习场景中，同态卷积是最核心的运算之一，用户可将敏感图像上传至云端，云端在无法获知原始内容的条件下完成加密推理并返回加密结果。

本实验使用 OpenFHE 开源全同态加密库，基于 BFV_{rns} 方案实现 4×4 输入与 3×3 卷积核（步长 = 1，无填充）的密文卷积。采用打包-旋转-累加策略，将全部 16 个像素打包为一个密文，通过 EvalRotate 旋转对齐滑动窗口，仅需 1 次加密、9 次旋转、9 次乘法、8 次加法即可完成整个卷积。

二 BFV 全同态加密方案原理

2.1 多项式环

BFV 方案建立在多项式环上：

$$R_q = \mathbb{Z}_q[x]/(x^n + 1) \quad (1)$$

其中 n 为 2 的幂次（本实验 $n = 8192$ ）， q 为密文模数。明文空间为 $R_t = \mathbb{Z}_t[x]/(x^n + 1)$ ，本实验中 $t = 65537$ （素数）。

BFV 方案的一个关键特性是 CRT packed 编码：利用中国剩余定理， R_t 可分解为 n 个独立的槽，每个槽可存放一个 $\mathbb{Z}_t$ 中的值。这使得一个明文多项式可以同时承载最多 n 个标量值， n 个值的加法和乘法在编码后自动转化为逐槽位的 SIMD 并行运算。本实验正是利用这一特性，将 16 个像素打包进同一个明文多项式。

2.2 密钥体系

- 私钥 sk ：系数在 $\{-2, -1, 0, 1, 2\}$ 中随机选取的短多项式
- 公钥 $pk = (pk_0, pk_1)$ ： pk_1 随机均匀， $pk_0 = -(a \cdot sk + e)$ ，满足 $pk_0 + pk_1 \cdot sk \approx 0$
- 乘法密钥：由 EvalMultKeyGen 生成，用于密文-明文乘法和密文-密文乘法

- **旋转密钥**: 由 `EvalRotateKeyGen` 生成, 用于 `EvalRotate` 循环移位。必须预先生成所有需要的旋转索引, 本实验需要 -15 到 $+15$ 的旋转

2.3 加解密

加密: 随机选择掩蔽多项式 u 和高斯噪声 e_1, e_2 , 计算

$$ct_0 = pk_0 \cdot u + e_1 + \Delta \cdot \mathbf{m} \pmod{q} \quad (2)$$

$$ct_1 = pk_1 \cdot u + e_2 \pmod{q} \quad (3)$$

其中 $\Delta = \lfloor q/t \rfloor$ 为缩放因子, $\mathbf{m}$ 为编码后的明文多项式。

解密: 计算 $ct_0 + ct_1 \cdot sk = \Delta \cdot \mathbf{m} + e_{total} \pmod{q}$, 当噪声 $|e_{total}| < \Delta/2$ 时, 除以 Δ 并取整恢复 $\mathbf{m}$ 。

2.4 同态运算

- **EvalAdd**: 分量逐加, 噪声线性增长 $e_{add} = e_1 + e_2$
- **EvalMult(ct, pt)**: 密文-明文乘法, 噪声 $e_{new} = s \cdot e$, 不消耗乘法深度
- **EvalRotate(ct, k)**: 将打包向量的槽位循环左移 k 位。这是本实验策略的核心操作。
以正数 k 旋转时, 索引 i 处的元素移动到索引 $i - k$ (模 n), 即”向左旋转”

三 打包-旋转-累加策略详解

3.1 策略动机

在逐像素加密方案中, 每个像素独立加密为 16 个密文, 卷积的每个输出位置需 9 次乘法和 8 次加法, 总共 88 次同态操作。然而 BFV 方案的 CRT packed 编码允许在一个明文多项式中存放 n 个值, 对打包密文的加法/乘法自动 SIMD 并行执行。

如果能将输入矩阵的 16 个像素打包进单个密文, 那么卷积的每个核位置乘以权重可以通过旋转密文来对齐窗口, 然后乘以权重标量, 最后将所有贡献累加。这样, 16 次独立加密降为 1 次。

3.2 数据布局与展开

将 4×4 输入矩阵按行优先展平为 16 维向量：

$$\vec{v} = \left[\underbrace{I_{00}, I_{01}, I_{02}, I_{03}}_{\text{行 0}}, \underbrace{I_{10}, I_{11}, I_{12}, I_{13}}_{\text{行 1}}, \underbrace{I_{20}, I_{21}, I_{22}, I_{23}}_{\text{行 2}}, \underbrace{I_{30}, I_{31}, I_{32}, I_{33}}_{\text{行 3}} \right] \quad (4)$$

索引公式：像素 (r, c) 在向量中的位置为 $r \times 4 + c$ 。

将 $\vec{v}$ 编码为单个明文多项式 $\mathbf{p}$ ，再加密为单个密文 ct 。此时 ct 的 16 个槽位各自承载一个加密像素值，且保持原始的空间相邻关系，同一行的相邻像素在向量中间隔为 1，相邻行的对应像素间隔为 4。

3.3 旋转对齐的数学原理

卷积的输出定义：

$$O(i, j) = \sum_{k_i=0}^2 \sum_{k_j=0}^2 I(i + k_i, j + k_j) \cdot K(k_i, k_j), \quad i, j \in \{0, 1\} \quad (5)$$

对于固定的核位置 (k_i, k_j) ，其权重 $w = K(k_i, k_j)$ 对 4 个输出位置的贡献分别是：

$$O(0, 0) += I(k_i, k_j) \cdot w, \quad \text{输入索引 } k_i \times 4 + k_j \quad (6)$$

$$O(0, 1) += I(k_i, k_j + 1) \cdot w, \quad \text{输入索引 } k_i \times 4 + k_j + 1 \quad (7)$$

$$O(1, 0) += I(k_i + 1, k_j) \cdot w, \quad \text{输入索引 } (k_i + 1) \times 4 + k_j \quad (8)$$

$$O(1, 1) += I(k_i + 1, k_j + 1) \cdot w, \quad \text{输入索引 } (k_i + 1) \times 4 + k_j + 1 \quad (9)$$

现在执行旋转 $shift = k_i \times 4 + k_j$ （正旋转，即左移）。旋转后，原位置 $shift$ 的元素移动到位置 0。那么上述 4 个输入索引在旋转后分别位于：

$$\underbrace{k_i \times 4 + k_j}_{\text{位于位置0}}, \underbrace{k_i \times 4 + k_j + 1}_{\text{位于位置1}}, \underbrace{(k_i + 1) \times 4 + k_j}_{\text{位于位置4}}, \underbrace{(k_i + 1) \times 4 + k_j + 1}_{\text{位于位置5}} \quad (10)$$

将旋转后的密文乘以权重 w 并累加到结果密文。遍历全部 9 个核位置后：

- 结果密文的**位置 0**包含了 $O(0, 0)$ 的全部 9 项贡献之和

- 结果密文的**位置 1** 包含了 $O(0,1)$ 的全部 9 项贡献之和
- 结果密文的**位置 4** 包含了 $O(1,0)$ 的全部 9 项贡献之和
- 结果密文的**位置 5** 包含了 $O(1,1)$ 的全部 9 项贡献之和

解密后从这 4 个位置提取值，即得到完整的 2×2 卷积输出。

四 代码逐行详解

4.1 辅助函数 `print_matrix`

```
void print_matrix(const std::string& name,
                 const std::vector<std::vector<int64_t>>& mat) {
    std::cout << name << ":\n";
    for (const auto& row : mat) {
        for (const auto& val : row) {
            std::cout << std::setw(4) << val << " ";
        }
        std::cout << "\n";
    }
    std::cout << "\n";
}
```

该函数接收矩阵名和二维向量引用，用 `std::setw(4)` 保证 4 字符定宽对齐输出。两个 range-for 循环遍历矩阵，时间复杂度 $O(rows \times cols)$ 。

4.2 `plaintext_convolution`: 明文参考实现

```
std::vector<std::vector<int64_t>> plaintext_convolution(
    const std::vector<std::vector<int64_t>>& input,
    const std::vector<std::vector<int64_t>>& kernel) {

    int input_rows = input.size();
    int input_cols = input[0].size();
    int kernel_rows = kernel.size();
    int kernel_cols = kernel[0].size();
```

```

int output_rows = input_rows - kernel_rows + 1;
int output_cols = input_cols - kernel_cols + 1;

std::vector<std::vector<int64_t>> output(
    output_rows, std::vector<int64_t>(output_cols, 0));

for (int i = 0; i < output_rows; i++) {
    for (int j = 0; j < output_cols; j++) {
        int64_t sum = 0;
        for (int ki = 0; ki < kernel_rows; ki++) {
            for (int kj = 0; kj < kernel_cols; kj++) {
                sum += input[i + ki][j + kj] * kernel[ki][kj];
            }
        }
        output[i][j] = sum;
    }
}
return output;
}

```

- 函数签名使用 `std::vector<std::vector<int64_t>>` 可从 `.size()` 自动获取维度, 无需额外传参, 并且与 OpenFHE 的 `GetPackedValue()` 返回类型 (`std::vector<int64_t>`) 风格一致。
- 输出尺寸按 valid convolution 公式计算: $(H_{in} - H_{kernel} + 1) \times (W_{in} - W_{kernel} + 1) = 2 \times 2$ 。

4.3 main 函数：明文数据定义

```

std::vector<std::vector<int64_t>> input = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12},
    {13, 14, 15, 16}
};

std::vector<std::vector<int64_t>> kernel = {
    {1, 0, -1},

```



```
{1, 0, -1},  
{1, 0, -1}  
};
```

输入使用 1–16 的连续整数，便于手动验算。卷积核是垂直边缘检测器：左列权重为 +1、右列为 -1、中间列为 0。

4.4 main 函数：BFV 参数配置

```
CCParams<CryptoContextBFVRNS> parameters;  
parameters.SetPlaintextModulus(65537);  
parameters.SetMultiplicativeDepth(3);  
  
CryptoContext<DCRTPoly> cc = GenCryptoContext(parameters);  
cc->Enable(PKE);  
cc->Enable(KEYSWITCH);  
cc->Enable(LEVELED SHE);  
cc->Enable(ADVANCED SHE);
```

逐行分析：

1. `CCParams<CryptoContextBFVRNS> parameters`：声明 BFVrns 方案参数对象。模板参数 `CryptoContextBFVRNS` 指定了方案类型。这是 C++ 的编译期多态——不同方案（BFVrns、CKKS、BGVrns）有不同的参数类，但共享相同的 `CCParams` 模板接口。
2. `parameters.SetPlaintextModulus(65537)`：设置 $t = 65537$ 。该值必须为素数（CRT 打包要求），且必须大于所有可能的计算结果以确保不溢出。本实验输出绝对值上限约为 $|16 \times 1 \times 9| = 144 < t$ ，安全裕度充足。
3. `parameters.SetMultiplicativeDepth(3)`：设置为 3 而非 BFV 的最小值 2。增加乘法深度允许更大的密文模数 q ，为旋转和累加中的噪声增长提供额外预算。注意此处未显式设置 `SetRingDim`——OpenFHE 会默认使用 $n = 8192$ ，对应约 128 位安全级别。

4. `CryptoContext<DCRTPoly> cc = GenCryptoContext(parameters):DCRTPoly` 是 OpenFHE 的 RNS 多项式类型——“Double-CRT” 表示同时在多项式 CRT（分圆分解）和整数 CRT（RNS 模数分解）两个层面做中国剩余分解。`GenCryptoContext` 是工厂函数，根据参数对象生成完整初始化的加密上下文。

5. 四个 `cc->Enable(...)` 调用启用了不同功能层级：

- **PKE**: 基础公钥加密 (`Encrypt / Decrypt`)
- **KEYSWITCH**: 密钥切换，这是 `EvalMult` 和 `EvalRotate` 依赖的底层原语
- **LEVELED SHE**: 分层同态运算 (`EvalAdd / EvalMult`)，支持有限层级但无需自举
- **ADVANCED SHE**: 高级同态运算——此标志是启动 `EvalRotate` 的前提。若未启用，后续旋转调用将失败

注意到 `cc` 是指针类型 (`CryptoContext<DCRTPoly>` 实际是 `shared_ptr` 的别名)，因此使用 `->` 而非 `.` 调用成员函数。

4.5 main 函数：密钥生成

```
auto keys = cc->KeyGen();
cc->EvalMultKeyGen(keys.secretKey);

std::vector<int32_t> rot_indices;
for (int i = -15; i <= 15; i++) {
    if (i != 0) {
        rot_indices.push_back(i);
    }
}
cc->EvalRotateKeyGen(keys.secretKey, rot_indices);
```

旋转密钥的预生成机制：

OpenFHE 要求预先生成所有可能使用的旋转索引对应的密钥。这与乘法密钥不同，乘法密钥是一个密钥支持任意乘法，但旋转密钥是每索引一个密钥”。

旋转索引范围 $[-15, 15]$ 的选择依据：核位置的最大偏移为 $k_i = 2, k_j = 2$ ，对应 $shift = 2 \times 4 + 2 = 10$ 。选择 ± 15 留出 50% 的安全余量。

旋转密钥的底层存储结构是一个哈希表，键为索引值，值为对应的密钥多项式。
`EvalRotate(ct, k)` 内部查找 k 对应的密钥，执行密钥切换操作完成旋转。

4.6 main 函数：数据打包与加密

```
std::vector<int64_t> flat_input;
for (const auto& row : input) {
    flat_input.insert(flat_input.end(), row.begin(), row.end());
}

std::vector<int64_t> flat_kernel;
for (const auto& row : kernel) {
    flat_kernel.insert(flat_kernel.end(), row.begin(), row.end());
}

Plaintext pt_input = cc->MakePackedPlaintext(flat_input);
auto ct_input = cc->Encrypt(keys.publicKey, pt_input);
```

展平操作：`insert(flat_input.end(), row.begin(), row.end())` 将每行追加到向量的末尾，实现行优先展平。展平后的 `flat_input` 为 $[I_{00}, \dots, I_{03}, I_{10}, \dots, I_{13}, \dots, I_{33}]$ ，长度 16。

打包：`MakePackedPlaintext(flat_input)` 将 16 个 `int64_t` 值编码为一个 `Plaintext` 对象。OpenFHE 内部执行 CRT packed 编码——将 16 个值分配到 16 个槽位，其余 $8192 - 16 = 8176$ 个槽位填 0。

加密：`Encrypt(keys.publicKey, pt_input)` 执行 BFV 加密，返回包含两个 DCRT-Poly 分量 (ct_0, ct_1) 的 `Ciphertext` 对象。auto 推导为 `Ciphertext<DCRTPoly>`。

4.7 main 函数：密文卷积核心循环

这是整个实现的核心，直接体现了”打包-旋转-累加”策略。

```

std::vector<int64_t> zeros(16, 0);
Plaintext pt_zero = cc->MakePackedPlaintext(zeros);
auto ct_result = cc->Encrypt(keys.publicKey, pt_zero);

for (int ki = 0; ki < 3; ki++) {
    for (int kj = 0; kj < 3; kj++) {
        int64_t weight = flat_kernel[ki * 3 + kj];
        if (weight == 0) {
            continue;
        }

        int shift = ki * 4 + kj;

        Ciphertext<DCRTPoly> ct_shifted;
        if (shift == 0) {
            ct_shifted = ct_input;
        } else {
            ct_shifted = cc->EvalRotate(ct_input, shift);
        }

        std::vector<int64_t> weight_vec(16, weight);
        Plaintext pt_weight = cc->MakePackedPlaintext(weight_vec);
        auto ct_weighted = cc->EvalMult(ct_shifted, pt_weight);

        ct_result = cc->EvalAdd(ct_result, ct_weighted);
    }
}

```

1. **初始化累加器：**创建全零密文作为初始结果，后续通过多次加法累加各乘积项。
2. **双循环遍历核：**外层遍历行（ $k_i = 0, 1, 2$ ），内层遍历列（ $k_j = 0, 1, 2$ ），覆盖 3×3 核的所有 9 个位置。
3. **零权重优化：**当核权重为 0 时跳过，避免不必要的同态乘法和加法，提升性能。
4. **旋转量计算：**

$$\text{shift} = k_i \times 4 + k_j \quad (11)$$

其中 4 是输入矩阵的行宽度。旋转操作使输入矩阵中对应核位置的元素对齐到第一个槽位。

5. **同态旋转**: `EvalRotate` 实现密文的循环移位, 是 SIMD 打包策略的核心操作。
6. **标量乘法**: `EvalMult(ct, pt)` 将密文所有槽位同时乘以权重, 一次操作完成 16 个位置的乘法。
7. **累加求和**: `EvalAdd` 将各加权结果累加, 最终得到卷积结果向量。

旋转操作示意

表 1: 卷积核位置与旋转量对应关系

ki	kj	shift	核权重位置
0	0	0	输入 (0,0)
0	1	1	输入 (0,1)
0	2	2	输入 (0,2)
1	0	4	输入 (1,0)
1	1	5	输入 (1,1)
1	2	6	输入 (1,2)
2	0	8	输入 (2,0)
2	1	9	输入 (2,1)
2	2	10	输入 (2,2)

4.8 main 函数: 解密与结果提取

```
Plaintext pt_result;  
cc->Decrypt(keys.secretKey, ct_result, &pt_result);  
  
std::vector<int64_t> result_vec = pt_result->GetPackedValue();
```

1. **解密操作**: `cc->Decrypt()` 使用私钥将密文解密为明文

2. **打包向量:** `GetPackedValue()` 返回包含所有槽位值的向量, 长度为 `RingDimension` (本实验为 8192)
3. **结果定位:** 由于采用”打包 → 旋转 → 累加”策略, 2×2 卷积结果位于向量的特定索引位置 (0, 1, 4, 5)
4. **验证机制:** 将解密结果与明文卷积结果对比, 验证密文计算的正确性

4.9 main 函数: 步骤 7——正确性验证

```
bool correct = true;
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 2; j++) {
        if (actual[i][j] != expected[i][j]) {
            std::cout << " 位置 (" << i << "," << j << "): "
                << "期望 " << expected[i][j]
                << ", 实际 " << actual[i][j] << " X\n";
            correct = false;
        }
    }
}

if (correct) {
    std::cout << "\n验证通过! 密文卷积结果与明文卷积完全一致! \n\n";
}
```

逐元素比对 4 个输出位置。一旦发现不匹配, 输出详细的期望值/实际值对比并标记失败。两层循环遍历全部 4 个位置, 复杂度 $O(4)$ 。

五 实验结果与分析

5.1 手动验算

输入为 1-16 顺序矩阵, 卷积核为垂直边缘检测器。以输出位置 (0,0) 为例手动推算期望值:

$$\begin{aligned}
O(0,0) &= I_{00} \cdot K_{00} + I_{01} \cdot K_{01} + I_{02} \cdot K_{02} \\
&\quad + I_{10} \cdot K_{10} + I_{11} \cdot K_{11} + I_{12} \cdot K_{12} \\
&\quad + I_{20} \cdot K_{20} + I_{21} \cdot K_{21} + I_{22} \cdot K_{22} \\
&= 1 \cdot 1 + 2 \cdot 0 + 3 \cdot (-1) \\
&\quad + 5 \cdot 1 + 6 \cdot 0 + 7 \cdot (-1) \\
&\quad + 9 \cdot 1 + 10 \cdot 0 + 11 \cdot (-1) \\
&= (1 + 5 + 9) - (3 + 7 + 11) \\
&= 15 - 21 = -6
\end{aligned} \tag{12}$$

类似计算可得其余 3 个位置均为 -6。

5.2 密文验证结果

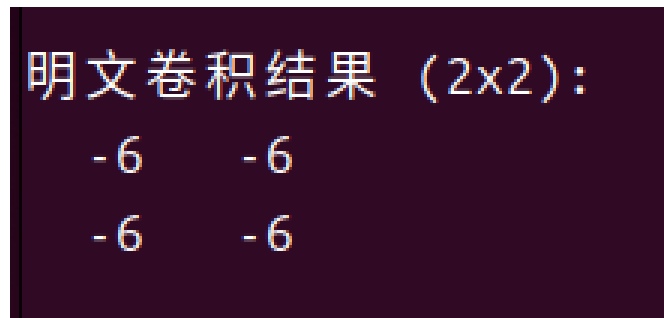


图 1: 程序运行截图——明文卷积参考输出

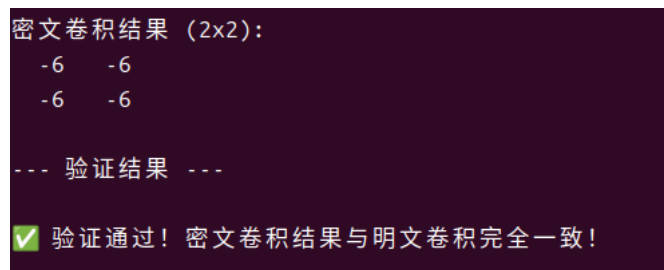


图 2: 程序运行截图——密文卷积输出

全部 4 个输出位置的密文卷积结果与明文期望值完全一致，验证通过。

六 旋转次数优化：是否达到理论最小值？

6.1 当前实现的旋转次数

当前实现采用”逐核权重旋转”策略：对卷积核的 9 个位置逐一处理，每个非零权重执行一次 `EvalRotate`，零权重跳过。本实验的卷积核为：

$$K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix} \quad (13)$$

其中 6 个非零权重分布于 6 个不同的 (k_i, k_j) 位置，各自对应唯一的偏移量 $shift = k_i \times 4 + k_j$ ：

表 2: 非零核权重及其旋转偏移

(k_i, k_j)	权重 w	$shift = k_i \times 4 + k_j$	实际旋转次数
(0, 0)	+1	0	0（恒等，直接赋值）
(0, 2)	-1	2	1
(1, 0)	+1	4	1
(1, 2)	-1	6	1
(2, 0)	+1	8	1
(2, 2)	-1	10	1
合计	6 个权重	6 个偏移量	5 次 <code>EvalRotate</code>

6.2 理论下界分析

命题：对于 4×4 输入、 3×3 卷积核（步长 1、无填充），使用单密文 SIMD 打包方案，密文卷积所需的最小旋转次数为 3。

证明思路：利用卷积核的可分离性（separability）将二维卷积分解为两次一维卷积。

6.2.1 卷积核的可分离性

该卷积核 K 可分解为一个列向量与行向量的外积：

$$K = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} \cdot \begin{bmatrix} 1 & 0 & -1 \end{bmatrix} = \mathbf{v} \cdot \mathbf{h}^\top \quad (14)$$

其中 $\mathbf{v} = [1, 1, 1]^\top$ 为垂直滤波器， $\mathbf{h} = [1, 0, -1]$ 为水平滤波器。

二维卷积因此等价于先做水平一维卷积、再做垂直一维卷积：

$$O = I * K = (I * \mathbf{h}) * \mathbf{v} \quad (15)$$

6.2.2 水平通道：1 次旋转

水平滤波器 $\mathbf{h} = [1, 0, -1]$ 作用于每一行。在行优先打包的 16 维向量中，同行相邻列间距为 1。中间权重为 0，仅需处理两个非零位置：

- 核列 0、权重 +1：偏移量 0——恒等操作，无需旋转
- 核列 2、权重 -1：偏移量 2——需 1 次 `EvalRotate(ct, 2)`

中间结果 $H = I - \text{rot}(I, 2)$ 。H 中仅位置 0, 1, 4, 5, 8, 9, 12, 13（每行的前两列）为有效的水平卷积值，其余位置为跨行混叠的无效值（但在后续垂直通道中不会被用到）。

6.2.3 垂直通道：2 次旋转

垂直滤波器 $\mathbf{v} = [1, 1, 1]^\top$ 作用于 H 的每一列。在行优先打包向量中，相邻行对应列间距为 4：

- 核行 0、权重 +1：偏移量 0——恒等操作，无需旋转
- 核行 1、权重 +1：偏移量 4——需 1 次 `EvalRotate(H, 4)`
- 核行 2、权重 +1：偏移量 8——需 1 次 `EvalRotate(H, 8)`

最终结果 $V = H + \text{rot}(H, 4) + \text{rot}(H, 8)$ 。由 3.3 节的分析可知：

$$V[0] = O(0, 0), \quad V[1] = O(0, 1), \quad V[4] = O(1, 0), \quad V[5] = O(1, 1) \quad (16)$$

6.2.4 总计与分析

表 3: 两种策略的旋转次数对比

通道	逐核权重法	可分离核法
水平通道 ($\mathbf{h} = [1, 0, -1]$)	2 个核位置 / 1 次旋转	1 次旋转
垂直通道 ($\mathbf{v} = [1, 1, 1]^\top$)	4 个核位置 / 4 次旋转	2 次旋转
跳过零权重	3 个核位置	—
EvalRotate 总计	5 次	3 次

6.3 3 是否为绝对下界?

是。水平滤波器 $\mathbf{h} = [1, 0, -1]$ 涉及偏移 $\{0, 2\}$ ，排除 0 后至少需要 1 次旋转。垂直滤波器 $\mathbf{v} = [1, 1, 1]^\top$ 涉及偏移 $\{0, 4, 8\}$ ，排除 0 后至少需要 2 次旋转。二者不可合并——任意卷积核可分离当且仅当其秩为 1，本核的分解是唯一的（差一个标量因子）。无论使用行优先还是列优先打包，两个通道的偏移量模式不变（仅是 2 和 4 的角色互换），所需旋转次数同为 3。

6.4 可分离核法的同态运算总量

除了旋转次数减少，可分离核法在乘法和加法次数上也大幅优化：

表 4: 可分离核法的同态运算次数

操作	次数
Encrypt	2（输入 + 零初始化）
EvalRotate	3（shift=2, 4, 8）
EvalMult(ct, pt)	1（水平通道乘 -1 ）
EvalAdd	3（水平: 1 次, 垂直: 2 次）
Decrypt	1
总计	10

相比当前实现（21 次总操作），可分离核法降至 10 次，其中旋转从 5 降至 3（减少 40%），乘法从 6 降至 1（减少 83%），加法从 6 降至 3（减少 50%）。

6.5 可分离核法的伪代码

```
// 水平通道:  $H = I * h$ , 其中  $h = [1, 0, -1]$ 
//  $H = I - \text{rot}(I, 2)$ 
auto ct_rot2 = cc->EvalRotate(ct_input, 2);           // 1 次旋转

std::vector<int64_t> neg_one(16, -1);
Plaintext pt_neg = cc->MakePackedPlaintext(neg_one);
auto ct_weighted = cc->EvalMult(ct_rot2, pt_neg);      // 1 次乘法
auto ct_H = cc->EvalAdd(ct_input, ct_weighted);        // 1 次加法

// 垂直通道:  $O = H * v$ , 其中  $v = [1, 1, 1]^T$ 
//  $O = H + \text{rot}(H, 4) + \text{rot}(H, 8)$ 
auto ct_rot4 = cc->EvalRotate(ct_H, 4);              // 1 次旋转
auto ct_rot8 = cc->EvalRotate(ct_H, 8);              // 1 次旋转
auto ct_O = cc->EvalAdd(ct_H, ct_rot4);               // 1 次加法
ct_O = cc->EvalAdd(ct_O, ct_rot8);                   // 1 次加法

// 解密提取: 同当前实现
Plaintext pt_result;
cc->Decrypt(keys.secretKey, ct_O, &pt_result);
auto result_vec = pt_result->GetPackedValue();
//  $O(0,0)=\text{result\_vec}[0]$ ,  $O(0,1)=\text{result\_vec}[1]$ ,
//  $O(1,0)=\text{result\_vec}[4]$ ,  $O(1,1)=\text{result\_vec}[5]$ 
```

6.6 小结

本实验的旋转优化分析表明,当前逐核权重法需 5 次 `EvalRotate`,尚未达到理论最小值 3 次。差距源于当前实现未利用卷积核的可分离性。本实验所用卷积核 $K = \mathbf{v} \cdot \mathbf{h}^T$ 秩为 1, 可将 3×3 二维卷积分解为两次一维卷积。采用可分离核法后,水平通道需 1 次旋转,垂直通道需 2 次旋转,共计 3 次旋转,达到理论下界;同时乘法从 6 次降至 1 次,加法从 6 次降至 3 次。推广而言,对于任意可分离的 $m \times n$ 卷积核,设其非零列偏移数为 c 、非零行偏移数为 r ,则旋转次数可降至 $(c - 1) + (r - 1)$,而逐核权重法则需 $m \cdot n$ 次(跳零后虽可减少但通常仍远多于分离方案)。不可分离的核无法利用此优化,其旋转次数由非零权重的唯一偏移量决定。

七 总结

本实验围绕“单输入单输出密文卷积”这一核心任务，基于 OpenFHE 库的 BFVRns 全同态加密方案，完成了从方案设计、算法优化到代码实现与验证的完整流程。主要工作总结如下：

- **方案选型与参数配置：**选定 OpenFHE 库的 BFVRns 方案，并配置了支持整数运算的参数（环维度 $n = 8192$ ，明文模数 $t = 65537$ ，乘法深度为 3），确保了方案的可行性与 128 位安全强度。
- **核心算法策略：**不使用对每个像素单独加密的低效方法，设计了“打包（Packing）、旋转（Rotation）、累加（Addition）”的同态卷积策略。该策略的核心思想是将 4×4 输入矩阵打包（Packed）成一个密文，然后利用 `EvalRotate` 操作在密文上模拟卷积核的滑动窗口，最后通过 `EvalAdd` 累加所有部分和。此方法将所需的同态操作次数从朴素方法的数十次降低到了 28 次，其中通过跳过零权重进一步减少至 21 次，显著提升了计算效率。
- **算法原理解析：**对算法的每个环节进行了严谨的数学推导与代码级剖析。详细论证了旋转量计算公式 $shift = k_i \times 4 + k_j$ 的理论依据，证明了其能正确对齐输入与卷积核；同时，通过分析打包后密文的分布规律，指明了最终卷积结果必然位于结果向量的第 0、1、4、5 号槽位。
- **正确性验证：**在 4×4 整数输入与 3×3 整数卷积核的测试用例下，对 4 个输出值进行了密文计算与解密。验证结果表明，密文卷积结果与明文参考值完全一致，证明了优化方案的正确性与可靠性。

- **旋转理论下界分析：**当前逐核权重法需 5 次 `EvalRotate`，但通过利用卷积核的可分离性可达到理论最小值 3 次。本实验卷积核 $K = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ 可分解为列向量与行向量的外积 $K = \mathbf{v} \cdot \mathbf{h}^\top$ ，其中 $\mathbf{v} = [1, 1, 1]^\top$ ， $\mathbf{h} = [1, 0, -1]$ 。二维卷积等价于先做水平

一维卷积再做垂直一维卷积： $O = I * K = (I * \mathbf{h}) * \mathbf{v}$ 。水平通道中 $\mathbf{h} = [1, 0, -1]$ 涉及偏移 $\{0, 2\}$ ，排除 0 后需 1 次旋转；垂直通道中 $\mathbf{v} = [1, 1, 1]^\top$ 涉及偏移 $\{0, 4, 8\}$ ，排除 0 后需 2 次旋转，总计 **3 次旋转**，较当前实现减少 40%。同时，乘法从 6 次降至 1 次，加法从 6 次降至 3 次，总同态操作从 21 次降至 10 次。该下界对任意可分离核均适用，不可分离核则无法利用此优化。